

UNIVERSIDADE FEDERAL DE SÃO CARLOS

Centro de Ciências Exatas e de Tecnologia

ARQUITETURAS DE ALTO DESEMPENHO

Relatório da Prática 4: Rede em Mesh para Processamento de Imagens

Docente: Prof. Dr. Emerson Carlos Pedrino

Discentes

Enzo Dezem Alves

RA: 801743

Curso: Engenharia de Computação

Gustavo de Oliveira Gimenes

RA: 820759

Curso: Engenharia de Computação

Guilherme Bartoletti Oliveira

RA: 821881

Curso: Engenharia de Computação

Gustavo Cesar Bento Laurindo

RA: 821402

Curso: Engenharia de Computação

São Carlos, 25 de maio de 2026

1 Introdução

O processamento digital de imagens (PDI) em tempo real exige arquiteturas de hardware capazes de lidar com grandes volumes de dados sob restrições de latência. Uma abordagem moderna para a interconexão de módulos de processamento em chips (SoCs) é o uso de Redes em Chip (*Networks-on-Chip* - NoC), organizadas frequentemente em topologias de malha (*mesh*).

Este trabalho descreve a implementação de uma rede em malha 4x4 em Verilog, projetada para a aplicação de operações de morfologia matemática (Dilatação e Erosão) em imagens binárias. A arquitetura proposta utiliza um caminho de dados em serpente para percorrer a malha, destacando-se pela integração de núcleos de processamento (*cores*) acoplados a todos os 16 nós roteadores. Essa abordagem permite uma alta flexibilidade, incluindo a seleção em tempo de execução de qual linha da matriz será responsável por realizar o processamento. O objetivo principal é validar a eficiência da NoC na coordenação do fluxo de dados e na aplicação distribuída de algoritmos em *streaming*.

2 Desenvolvimento Teórico

2.1 Redes em Malha (Mesh NoC)

Uma NoC em malha consiste em uma grade de roteadores interconectados, onde cada roteador pode se comunicar com seus vizinhos nas direções Norte, Sul, Leste e Oeste, além de possuir uma porta Local para comunicação com seu respectivo núcleo de processamento (*core*). A roteação XY é um dos algoritmos mais comuns, onde os dados viajam primeiro na horizontal (X) e depois na vertical (Y) até o destino.

No projeto em questão, foi adotada uma variação de roteamento que descreve um caminho em serpente (*serpentine path*) através da malha 4x4. Este caminho garante que os dados trafeguem de forma ordenada e contínua pelo sistema, permitindo que a filtragem ocorra na linha de processamento previamente selecionada antes que os pacotes sejam extraídos na interface de saída.

2.2 Filtros de Máximo e Mínimo (Morfologia Matemática)

Os filtros de máximo e mínimo são operações morfológicas não lineares fundamentais no processamento digital de imagens. Operando especificamente sobre imagens binárias (onde cada pixel possui 1 bit), essas técnicas são utilizadas para extrair, refinar ou limpar componentes visuais, sendo classicamente conhecidas como operações de Dilatação e Erosão.

Em vez de calcular uma mediana, esses filtros analisam uma janela espacial (vizinhança) ao redor de um pixel central para definir o estado de saída:

- **Filtro de Mínimo (Erosão):** O valor de saída será o valor mínimo entre todos os vizinhos. Na lógica binária, o pixel de saída só será 1 se **todos** os pixels da janela forem 1. Sua função prática é "corroer" bordas e eliminar ruídos brancos isolados em fundos escuros. Matematicamente, operando sobre n pixels vizinhos (P_0 a P_{n-1}):

$$P_{out_min} = \min(P_0, P_1, \dots, P_{n-1}) = \bigwedge_{i=0}^{n-1} P_i \quad (1)$$

- **Filtro de Máximo (Dilatação):** O valor de saída será o valor máximo da vizinhança. O pixel de saída será 1 se **pelo menos um** vizinho for 1. Sua função é expandir objetos, preenchendo pequenos "buracos" pretos ou falhas.

$$P_{out_max} = \max(P_0, P_1, \dots, P_{n-1}) = \bigvee_{i=0}^{n-1} P_i \quad (2)$$

No hardware, especialmente integrados como núcleos (*cores*) em uma rede em malha, a implementação desses filtros atinge alta eficiência e baixa latência. O Filtro de Mínimo é mapeado diretamente por meio de uma porta lógica **AND** de múltiplas entradas englobando a janela em análise, enquanto o Filtro de Máximo é resolvido por uma porta lógica **OR** paralela.

3 Desenvolvimento Prático

A implementação foi dividida em três níveis hierárquicos principais: os nós de roteamento, os núcleos de processamento periféricos (*cores*) e as interfaces de empacotamento, todas instanciadas na malha *top-level*.

3.1 Arquitetura do Roteador

O módulo `router.v` é responsável por gerenciar o fluxo de pacotes na rede com base em suas coordenadas paramétricas (X, Y). O roteador possui cinco interfaces bidirecionais (Norte, Sul, Leste, Oeste e Local/Core). A lógica de roteamento implementa o Algoritmo XY estrito, priorizando o deslocamento no eixo X antes das correções no eixo Y.

A arquitetura utiliza um barramento de 6 bits, estruturado da seguinte forma: os bits [5:4] armazenam a coordenada X de destino, os bits [3:2] a coordenada Y, o bit [1] atua como validador (*Valid*) e o bit [0] transporta a carga útil (*Pixel*).

Para resolver colisões simultâneas nas portas de entrada, implementou-se uma lógica combinacional de arbitragem por prioridade fixa. A precedência máxima é dada aos dados provenientes do próprio núcleo de processamento (porta *Core*), seguidos pelo eixo

horizontal (Leste-Oeste) e, por último, pelo eixo vertical (Norte-Sul). A Listagem 1 ilustra essa hierarquia.

Listing 1: Trecho da lógica combinacional de arbitragem e roteamento XY.

```
1 // --- PACOTE VINDO DO OESTE (Viajando para a Direita) ---
2 if (in_west[1]) begin
3     // Verifica se a saída leste está livre (!next_out_east[1])
4     if (in_west[5:4] > X && !next_out_east[1]) begin
5         next_out_east = in_west;
6     end
7     else if (in_west[5:4] == X) begin
8         if (in_west[3:2] > Y && !next_out_north[1]) begin
9             next_out_north = in_west;
10        end else if (in_west[3:2] == Y && !next_out_core[1]) begin
11            next_out_core = in_west; // Destino alcançado
12        end
13    end
14 end
```

3.2 Interfaces de Rede (Empacotador e Desempacotador)

Para acoplar o fluxo contínuo de dados à NoC, os módulos packeger e despackeger atuam como *Network Interfaces* (NIs). O empacotador converte o bit de validação e o pixel bruto em um pacote endereçável, concatenando-os às coordenadas de destino estáticas da primeira fase da malha, conforme demonstrado na Listagem 2.

Listing 2: Lógica de empacotamento com endereçamento *hardcoded*.

```
1 // Coordenadas iniciais estaticas para injecao na malha
2 wire [1:0] dest_x = 2'b00;
3 wire [1:0] dest_y = 2'b00;
4
5 // Concatenacao do pacote de 6 bits
6 assign pack_out = {dest_x, dest_y, valid_in, pixel_in};
```

Na extração, o despackeger opera de forma combinacional direta, descartando o cabeçalho de rede (bits [5:1]) e isolando o pixel resultante para a reconstrução da imagem processada, minimizando o impacto na latência do sistema.

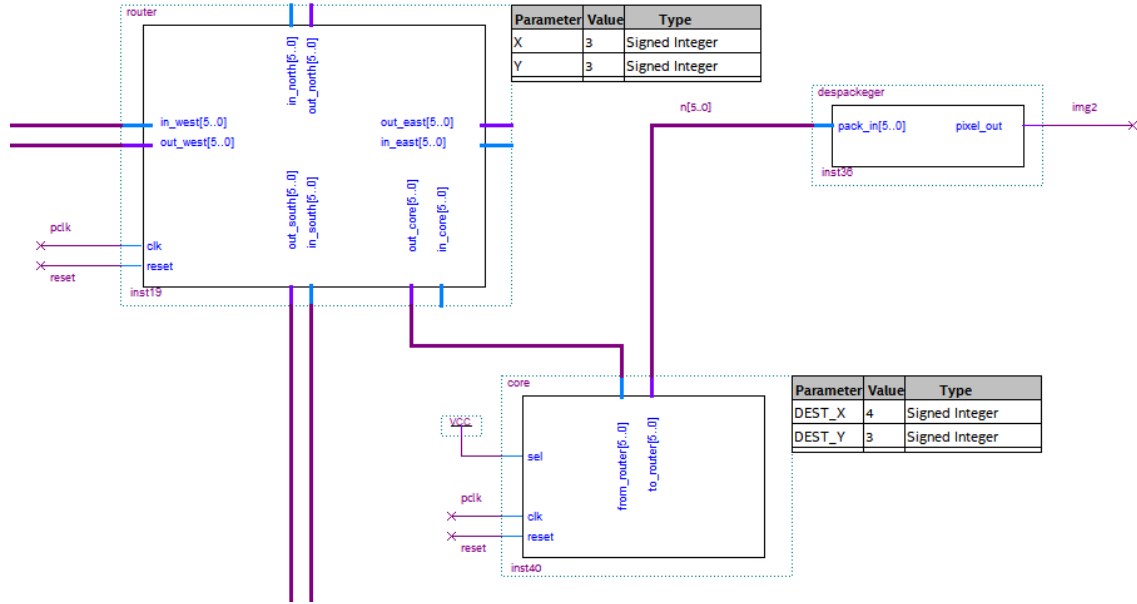


Figura 1: Diagrama de blocos do módulo desempacotador extraindo a carga útil.

3.3 Núcleo de Processamento (Core)

O módulo `core.v` foi desenvolvido para realizar as operações morfológicas requeridas. Ele implementa um filtro espacial bidimensional operando sobre uma janela móvel de 3×3 pixels. Para manter a coerência temporal dos dados e evitar o gargalo de múltiplas leituras em memória, a arquitetura utiliza registradores de deslocamento (*line buffers*) (Listagem 3).

Listing 3: Atualização dos *line buffers* e extração da janela 3×3 .

```

1  always @(posedge clk or negedge reset) begin
2      if (!reset) begin
3          reg0 <= 256'b0; reg1 <= 256'b0; reg2 <= 3'b0;
4      end else if (valid_in) begin
5          reg1 <= {reg1[254:0], img_in}; // Buffer da linha atual
6          reg0 <= {reg0[254:0], reg1[255]}; // Buffer da linha anterior
7          reg2 <= {reg2[1:0], reg0[255]}; // Buffer de transicao
8      end
9  end
10 // Janela espacial de 9 bits
11 assign window = {reg0[2:0], reg1[2:0], reg2};

```

A operação morfológica propriamente dita é calculada concorrentemente. O sinal de controle `sel` multiplexa a operação: nível lógico alto aplica o Filtro de Mínimo (Erosão via redução AND), enquanto o nível lógico baixo aplica o Filtro de Máximo (Dilatação via redução OR).

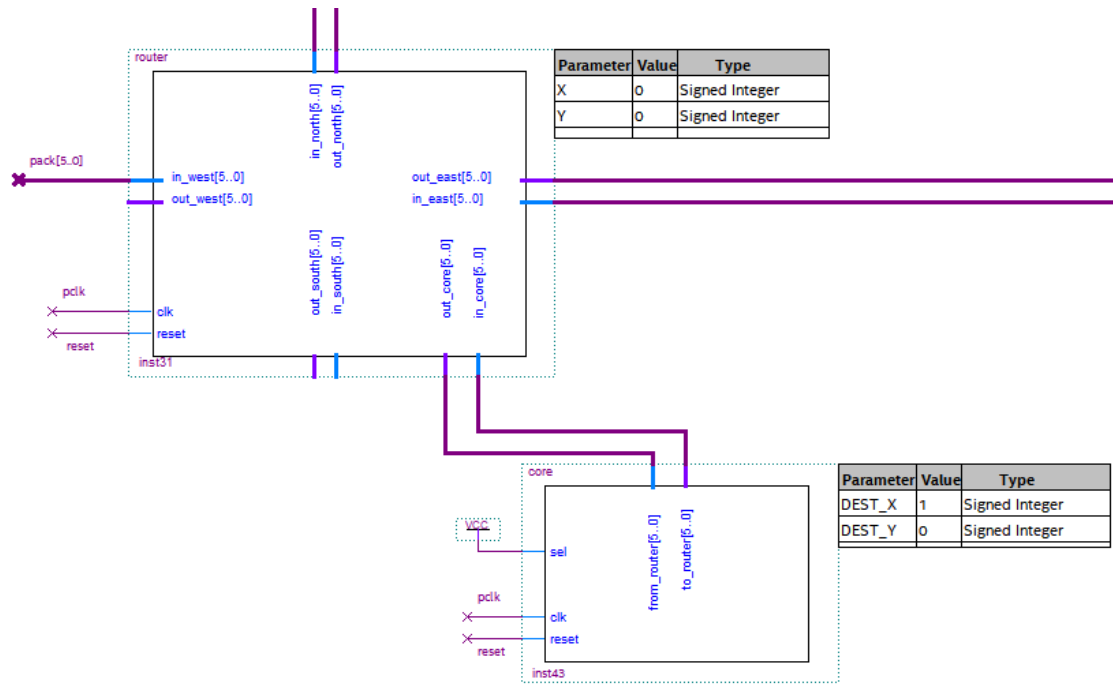


Figura 2: Integração estrutural entre o roteador e o núcleo de processamento.

3.4 Integração Top-Level

O módulo principal congrega a topologia *mesh* 4x4, instanciando os 16 nós roteadores, cada um acoplado ao seu respectivo núcleo de processamento (totalizando 16 *cores* distribuídos por toda a matriz), além da lógica periférica de injeção e extração. A arquitetura foi estruturada para suportar um fluxo contínuo e determinístico. Um de seus diferenciais operacionais é a flexibilidade do roteamento: o sistema permite configurar em qual linha da matriz (eixo Y) os pacotes serão efetivamente desviados para processamento. O *datapath* pode ser rastreado nas seguintes etapas:

1. **Injeção na Malha:** O empacotador sintetiza o cabeçalho inicial de roteamento e insere o pacote no nó de entrada (0,0).
2. **Propagação (Roteamento XY):** Os dados percorrem a malha prioritariamente ajustando a coordenada horizontal. A lógica de arbitragem garante a fluidez da comunicação, direcionando os dados até a coluna de destino antes do avanço vertical.
3. **Processamento:** Ao alcançar um roteador pertencente à linha previamente selecionada para a execução do algoritmo, o pacote é absorvido pela porta Local. O núcleo (*core*) associado àquele nó insere o pixel na matriz de *line buffers*, aplica a filtragem correspondente e atualiza o cabeçalho com as coordenadas do próximo destino no caminho.

4. **Extração:** Ao atingir o roteador definido como saída da malha, o pacote é encaminhado para o desempacotador, que disponibiliza o pixel tratado e a *flag* de valid para a montagem do *frame* de saída.

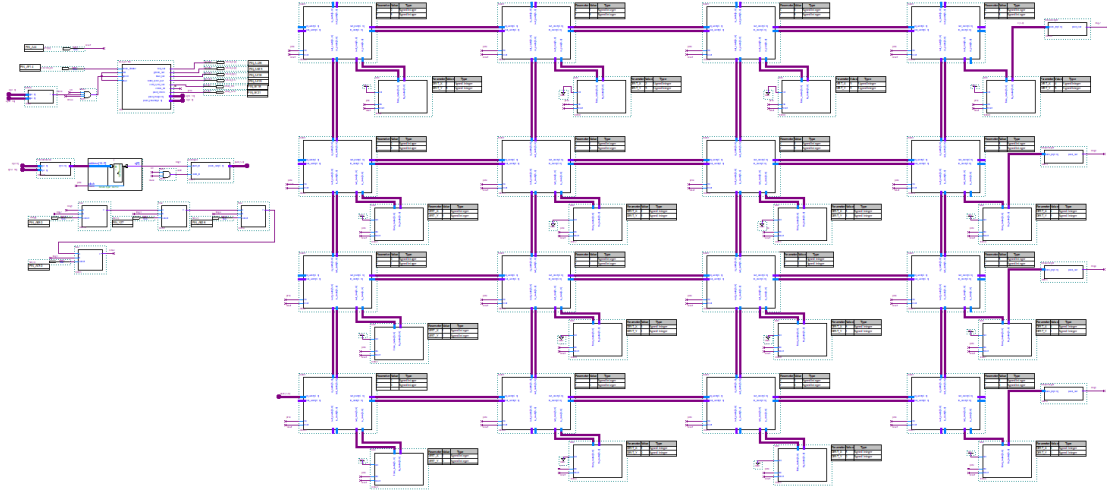
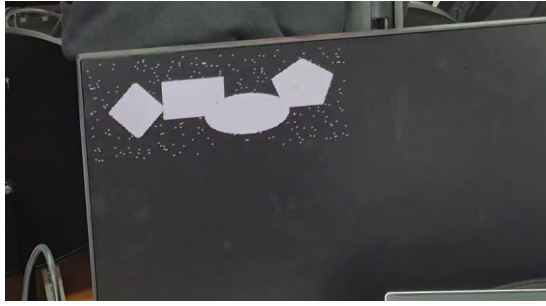


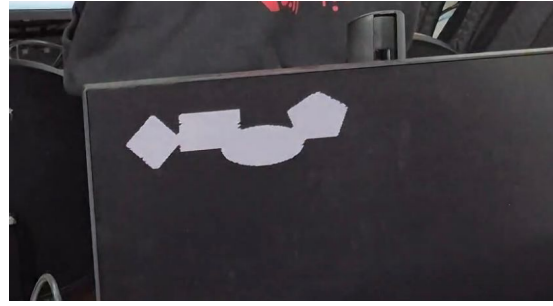
Figura 3: Esquemático do módulo Top-Level evidenciando as conexões da malha 4x4. Cada um dos 16 nós roteadores possui um núcleo de processamento integrado à sua porta Local.

4 Resultados e Discussão

Nesta seção, apresentam-se os resultados visuais e físicos obtidos a partir da implementação em hardware. A filtragem foi validada através da injeção de uma imagem corrompida por ruído na rede em malha.



(a) Imagem binária original corrompida por ruído.



(b) Imagem processada pelo filtro morfológico via NoC.



(c) Montagem experimental (*setup*) utilizada para a validação da arquitetura.

Figura 4: Comparativo visual do processamento de imagens e o ambiente físico de testes do sistema *Mesh PDI*.

5 Conclusão

A implementação da rede em malha 4x4 para processamento digital de imagens demonstrou o funcionamento adequado da arquitetura proposta, validando tanto o mecanismo de comunicação da NoC quanto a integração entre roteadores e núcleos de processamento distribuídos. O fluxo de dados em streaming foi corretamente coordenado pelo sistema de roteamento, permitindo o encaminhamento contínuo dos pacotes ao longo da malha.

Os filtros morfológicos de máximo (Dilatação) e mínimo (Erosão) foram aplicados corretamente sobre a imagem binária de entrada, resultando no tratamento esperado da imagem corrompida por ruído. As operações realizadas pelos cores possibilitaram a remoção de imperfeições e o refinamento das estruturas visuais presentes na imagem processada, comprovando a eficácia da arquitetura para aplicações de processamento digital de imagens em hardware.

Além disso, a utilização de uma NoC baseada em topologia mesh evidenciou vantagens relacionadas à modularidade, escalabilidade e paralelismo do sistema, tornando a solução adequada para aplicações embarcadas que demandam alto desempenho e baixa latência.

6 Referências

1. PEDRINO, E. C. Sistemas Digitais - Aula 01 (Introdução a PLDs). UFSCar, 2022.
2. DALLY, W. J.; TOWLES, B. P. **Principles and Practices of Interconnection Networks**. Morgan Kaufmann, 2004.
3. ROTEIRO DA PRÁTICA 4. Processamento de Imagens em Rede em Malha. Disciplina de Arquiteturas de Alto Desempenho, UFSCar, 2026.